marquis's final | Serverless text processing

AWS Architecture Portfolio Project | Category B | Marquis | October 25, 2026

Executive summary

Small teams that receive text documents need a repeatable way to count words and record results without running a server continuously. This project serves that need by turning each uploaded UTF-8 text file into a JSON summary and a database record labeled "marquis's final". The stack uses Amazon S3, AWS Lambda with automatic serverless scaling, Amazon DynamoDB, CloudWatch logs and an error alarm, IAM roles, and AWS CloudFormation.

Architecture and service decisions

An authenticated user uploads a .txt file into a private S3 input bucket. S3 sends an ObjectCreated notification to Lambda, which reads the object, decodes UTF-8 text, counts whitespace-separated words and characters, writes a JSON object to a separate results bucket, and writes a DynamoDB item keyed by source_file. CloudWatch records the execution and monitors the Lambda Errors metric. The diagram in Appendix A was committed before the corresponding deployment; the expanded diagram was committed before the stack update.

S3 was chosen over an EC2-attached disk because uploads can trigger processing directly and objects remain independent of a compute instance. Two buckets separate input and output permissions and prevent output writes from invoking the function recursively. S3 Standard is appropriate for immediate access to small active documents; archival storage would add retrieval complexity without a useful saving at this scale.

Lambda was chosen over EC2 or ECS because this task runs briefly after an upload. It automatically supplies execution capacity without a continuously running server, operating-system maintenance, or a load balancer. There is no public web application or API Gateway: the authenticated S3 upload is the entry point, and the working screenshots satisfy the assignment's alternative to a public URL.

DynamoDB stores a compact lookup record for each source filename. It was chosen over RDS because the access pattern is a single-key lookup, with no joins, relational schema, or persistent database instance required. On-demand capacity matches intermittent traffic. The S3 JSON copy remains useful as an export; the two writes are not an atomic transaction, which is a documented limitation.

CloudWatch was chosen over hosting a monitoring server because Lambda already integrates with its logs and metrics. IAM supplies temporary role credentials rather than embedding access keys. CloudFormation was chosen over manual console setup to make the complete stack repeatable and removable as one unit. These choices cover compute, serverless scaling, S3, a database, and CloudWatch alarms from the rubric.

Implementation and reproduction

Prerequisites are an AWS account with permission to manage the listed resources, an authenticated AWS CLI or CloudShell session, Python 3 for tests, and the repository files. Run commands from the repository root. The demonstration uses us-west-2 and stack name marquis-final-text. No credentials belong in the repository. The Lambda source is in src/index.py and is embedded in template.json so CloudFormation can deploy it without a separate code-package bucket.

1. Review architecture/architecture.drawio or architecture.svg. Check the source and IAM scope in template.json. Run python3 -m unittest discover -s tests to verify key decoding, word-count output, the database write payload, and rejection of objects larger than 1 MiB. These local tests use a fake AWS client and are separate from the live test.

2. Run bash deploy.sh. It verifies the signed-in identity, validates the template, deploys with CAPABILITY_IAM, and prints stack outputs. That acknowledgment permits the project execution role to be created. For another region or stack name, set AWS_REGION and STACK_NAME before running the scripts. Wait for CREATE_COMPLETE or UPDATE_COMPLETE before uploading a file.

3. Run bash test-live.sh. The script discovers bucket names from stack outputs, uploads sample.txt, waits for its JSON result, checks the expected count and project label, reads the DynamoDB result consistently, and displays recent execution logs. Open the resources in the AWS console and capture screenshots before cleanup. Upload only UTF-8 .txt files of 1 MiB or less; the word definition is Python whitespace splitting, not linguistic tokenization.

Key configuration: Python 3.12, 128 MB memory, a 30-second timeout, a .txt event suffix filter, S3 AES-256 server-side encryption, public-access blocking, and bucket policies denying non-TLS access. DynamoDB uses on-demand billing and encryption at rest. Lambda receives only GetObject for input objects, PutObject for result objects, PutItem for this table, and log-stream/write permissions. The invocation permission is restricted to this account and input bucket.

Challenges and solutions

The local AWS CLI returned IncompleteSignature, so deployment used the authenticated console and CloudShell instead. The full rubric arrived after the initial stack was created; the diagram was updated and committed before adding DynamoDB and the CloudWatch alarm. A notification dependency cycle was avoided by deriving the input bucket name from stack, account, and region and making creation depend on Lambda invoke permission. An initial local test expected 11 words; manual recount corrected that expectation to 10. A pasted test command also required a quoting correction before it could run.

Observed testing results

CloudFormation reached UPDATE_COMPLETE, and all ten stack resources showed completed states. The live S3 event produced 10 words and 76 characters; a consistent DynamoDB read returned matching values and the requested project label. CloudWatch recorded a successful invocation with 865.18 ms execution duration, 527.29 ms initialization, 1,393 ms billed duration, and 95 MB maximum memory used. The error alarm was observed in OK state. No load test, AZ outage, alarm-transition test, or recovery-time drill was performed.

Architectural concerns

Security. The compute code uses its IAM execution role and contains no access keys. Both buckets block public access, encrypt objects at rest, and deny non-HTTPS requests. DynamoDB is encrypted; AWS SDK service requests use TLS. This managed-service design has no EC2 instances, SSH listener, VPC attachment, or security groups to configure. Access is governed by IAM and resource policies. A production version should add least-privilege uploader/reader roles and avoid sensitive data in filenames or logs; the sample contains no private document content.

Reliability. S3 Standard stores objects across at least three Availability Zones; DynamoDB also replicates across three AZs, and Lambda uses multiple AZs. A single-AZ interruption should therefore be handled by the managed services, although this project did not test that failure. All application resources remain in one region, so this is not a regional failover design. Lambda retries asynchronous failures by default, but this stack has no dead-letter queue or failure destination. Exhausted retries can leave an uploaded object without a completed result.

Duplicate processing of an unchanged file overwrites the same result key and DynamoDB key rather than accumulating duplicate rows. This is useful but is not a guarantee of exactly-once processing. Overwriting an input filename while events are in flight can mix versions because the code reads the current object. A failure between the S3 and DynamoDB writes can temporarily leave only one result. Versioned input keys, a processing-state record, conditional writes, and an SQS failure path are concrete improvements.

Recovery objectives and DR strategy. For this demonstration, the proposed RTO is 30 minutes to redeploy from CloudFormation and re-upload locally retained source files; this is a planning target, not a measured result. The proposed RPO is zero for the included sample because its source is committed locally. For arbitrary production uploads, this deployment has no guaranteed RPO against accidental deletion or a regional disaster: versioning, point-in-time recovery, and cross-region backups are not enabled. Recovery would identify the failure, redeploy, restore available source copies, replay uploads, and compare both result stores. A production DR plan needs tested backups and a timed drill.

Performance. Lambda scales concurrent invocations as uploads arrive, within account quotas, while DynamoDB on-demand capacity adjusts to demand. The sample uses 95 MB within its 128 MB allocation. Bottlenecks include S3 read latency, cold starts, sequential writes, account concurrency limits, and the in-memory word split. The 1 MiB input limit bounds this demonstration's memory use. Larger documents would require streaming or chunking, and bursts would benefit from an SQS queue for controlled backpressure. One successful sample is not evidence of sustained throughput.

Cost. The saved AWS Pricing Calculator estimate for Oregon displays $0.14 per month and $0 upfront: S3 $0.03, CloudWatch $0.11, and Lambda and DynamoDB each round to $0.00. Assumptions are 1,000 invocations at 128 MB and 1 second, 1 GB combined S3 storage, 2,000 PUT and 2,000 GET requests, 0.01 GB DynamoDB storage with 1,000 writes and reads, and 0.01 GB logs with one standard alarm. Lambda free-tier discounts were excluded. Actual usage, retries, taxes, and additional transfers may differ; this is not a bill. A $1 monthly planning allowance is conservative for this small modeled workload.

Cost optimizations and operations

Applied optimization one: event-driven Lambda eliminates an always-on compute instance for an intermittent task. Applied optimization two: on-demand DynamoDB avoids paying for idle provisioned capacity. Seven-day CloudWatch retention also limits log accumulation. With longer-lived use, S3 lifecycle expiration would remove obsolete source and result files. The cost estimate and assumptions are in COST.md, with a saved calculator link and screenshot in evidence/.

Well-Architected self-review: three pillars

Security pillar - strength: the Lambda role is scoped to the specific input, output, database, and logging resources, and the buckets deny public and non-TLS access. Improvement: create separate uploader and result-reader roles, add policy checks to deployment, and review data retention and logging for real documents. Administrative deployment access is broader than runtime access and should not become the normal user workflow.

Reliability pillar - strength: regional managed services provide multi-AZ infrastructure and the application can be recreated from CloudFormation. Improvement: add SQS with a dead-letter queue, versioned source keys, DynamoDB point-in-time recovery, and backups appropriate to an explicit production RPO. Perform an actual recovery exercise and verify reconciliation after a partial write failure before claiming the 30-minute target.

Cost optimization pillar - strength: the workload uses brief, usage-based compute and database capacity, with an explicit Pricing Calculator estimate and short log retention. Improvement: add AWS Budgets alerts, lifecycle rules, and periodic reviews of actual usage. Measure memory and duration at realistic file sizes before changing Lambda memory, because the smallest allocation is not always the lowest total execution cost.

What I would do differently

I would start with the full rubric and workload requirements before building, so the database, alarm, cost assumptions, and evidence plan are agreed at the beginning. I would also give more time to failure handling than to the happy-path demonstration: queued events, a dead-letter queue, source version tracking, and a tested recovery procedure would make this useful beyond a classroom example. The current design demonstrates integration and least-privilege access, but one sample cannot establish production reliability or performance. I used assisted tooling to prepare and deploy the project; understanding and defending the service tradeoffs remains part of my review before submission.

Deployment cleanup

After capturing the working-system evidence, I emptied the two demonstration buckets and deleted the marquis-final-text stack in us-west-2. CloudFormation confirmed DELETE_COMPLETE at 2026-10-26 06:50 UTC (October 25, 11:50 PM Pacific). All ten stack resources, including both buckets, Lambda, DynamoDB, the alarm, log group, IAM role, invocation permission, and bucket policies, showed DELETE_COMPLETE. The demonstration deployment is no longer running. Source code and screenshots remain in the public repository; unrelated coursework resources were not targeted.

References and evidence

Saved AWS Pricing Calculator estimate

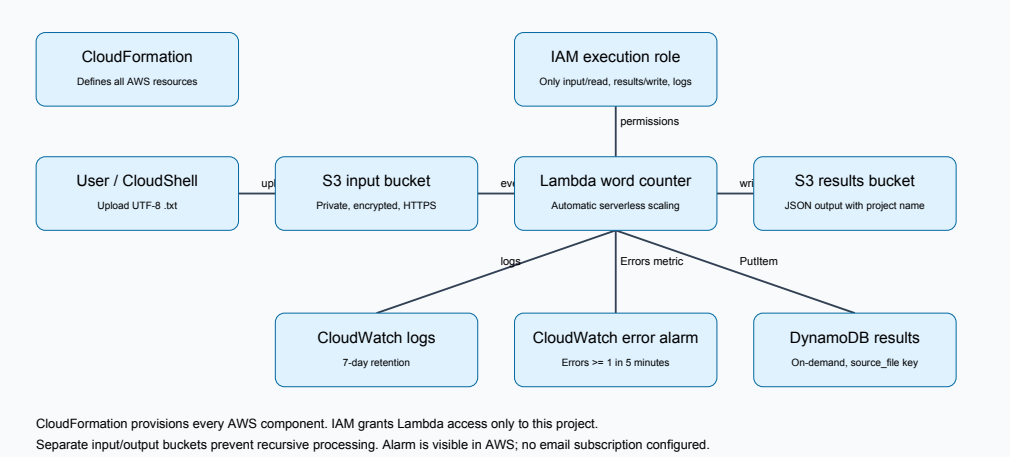
Lambda resilience

Lambda asynchronous retries

DynamoDB overview

S3 FAQ

Appendix A | Architecture diagram



Solid arrows show application data, permission association, and monitoring signals. CloudFormation defines every AWS component. The editable draw.io file additionally shows provisioning relationships. This appendix is excluded from the four-page written report.

Appendix B | Working system evidence



Authenticated AWS CloudShell output shows the deployed stack, project name, 10-word result, and matching DynamoDB count. The evidence folder also contains logs, deployed resources, alarm state, and the cost estimate screenshot.